

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

UDC XXXXXXXX

Research Project Report on the Topic:
A/B Testing of a Recommender System as a Black-Box Experiment
(interim, the first stage)

Submitted by the Student:

group #БПА/Д244, 2nd year of study Sapunov Artemiy Maximovich

Approved by the Project Supervisor:

Ramazyan Tigran Armenovich
Research Fellow
Faculty of Computer Science, HSE University

Contents

Abstract	3
Keywords	3
1 Introduction	4
1.1 Subject Area and Motivation	4
1.2 Research Question and Contribution	4
1.3 Planned Outcomes	4
2 Literature Review	5
2.1 Benchmark Suites for Black-Box Optimisation	5
2.2 Gradient-Based vs. Derivative-Free Optimisation	5
2.3 Symbolic Computation in Scientific Software	6
2.4 A/B Testing as Black-Box Optimisation	6
3 Methodology	7
3.1 The <code>blackbox_benchmarks</code> Package	7
3.2 Domain Coverage	8
3.3 Stochastic Benchmark: A/B Testing	8
3.4 Experimental Protocol	9
4 Experimental Results	10
4.1 Optimiser Comparison Across Domains	10
4.2 Function Evaluation Counts	10
4.3 A/B Testing Benchmark Results	10
4.4 Package Validation	11
5 Discussion	13
5.1 Answer to the Research Question	13
5.2 Role of SymPy in the Package	13
5.3 A/B Testing as a Black-Box Domain	13
5.4 Limitations	13
6 Plan of Further Work	15
6.1 Extended Optimiser Comparison	15
6.2 Noise Sensitivity Study	15
6.3 Constrained Benchmark Variants	15
6.4 Package Release	15
7 Conclusion	16
References	17

Abstract

This project investigates how the structural properties of black-box objective functions — such as smoothness, multimodality, dimensionality, and stochasticity — affect the performance of standard optimisation algorithms. The central research question is: *do gradient-based optimisers consistently outperform derivative-free methods on smooth, low-dimensional benchmarks, and does this advantage disappear on multimodal or stochastic functions?* To answer this question, a Python package `blackbox_benchmarks` was developed, providing 13 benchmark functions across 6 scientific domains (mathematics, physics, chemistry, engineering, economics, and online A/B testing). The package is built on the SymPy computer algebra system, enabling exact analytical gradients and Hessians for all deterministic benchmarks. Three optimisation algorithms — Nelder-Mead, L-BFGS-B, and BFGS — were applied to each benchmark and compared on success rate and number of function evaluations. Results show that gradient-based methods require significantly fewer evaluations on smooth, unimodal functions but fail consistently on multimodal landscapes, where derivative-free methods such as Nelder-Mead are more reliable.

Аннотация

Проект исследует влияние структурных свойств целевых функций типа «чёрный ящик» — гладкости, мультимодальности, размерности и стохастичности — на эффективность стандартных алгоритмов оптимизации. Центральный исследовательский вопрос: *превосходят ли методы на основе градиентов производные-свободные методы на гладких, низкоразмерных бенчмарках, и исчезает ли это преимущество на мультимодальных или стохастических функциях?* Для ответа на этот вопрос разработан Python-пакет `blackbox_benchmarks`, включающий 13 эталонных функций из 6 научных областей (математика, физика, химия, инженерия, экономика и онлайн А/В-тестирование). Пакет построен на системе компьютерной алгебры SymPy, обеспечивающей точные аналитические градиенты и гессианы для всех детерминированных бенчмарков. Три алгоритма оптимизации — Nelder-Mead, L-BFGS-B и BFGS — применялись к каждому бенчмарку и сравнивались по доле успешных запусков и числу обращений к функции. Результаты показывают, что градиентные методы требуют значительно меньше вычислений на гладких одномодальных функциях, однако систематически не справляются с мультимодальными ландшафтами, где более надёжным оказывается метод Нелдера-Мида.

Keywords

black-box optimisation; benchmark functions; derivative-free optimisation; gradient-based methods; SymPy; Python package; multimodality; A/B testing; recommender systems; Nelder-Mead; L-BFGS-B

1 Introduction

1.1 Subject Area and Motivation

Black-box optimisation addresses problems where the objective function cannot be inspected analytically — it can only be queried at chosen input points. This setting is ubiquitous: a single evaluation may require a full physical simulation, a real-world experiment, or a stochastic user interaction. Choosing the right optimisation algorithm for a given problem is therefore non-trivial, and the choice depends heavily on the structural properties of the function being optimised.

Benchmark functions are the standard tool for studying algorithm behaviour. A benchmark exposes a known ground truth (global minimum, analytical gradient) while modelling the difficulty characteristics of real problems. Existing suites such as BBOB/COCO [2] focus almost exclusively on mathematical test functions, leaving open the question of whether algorithm rankings generalise to physically or experimentally motivated objectives.

1.2 Research Question and Contribution

This project addresses the following research question:

Do gradient-based optimisation algorithms consistently outperform derivative-free methods on smooth, unimodal black-box benchmarks, and does this advantage disappear on multimodal or stochastic functions from real-world scientific domains?

To answer this question, we make two contributions:

1. We develop **blackbox_benchmarks**, a plug-and-play Python package providing 13 benchmark functions across 6 scientific domains. The package is built on SymPy [5], which provides exact analytical gradients and Hessians without any hand-coded derivatives. The package is available as a single self-contained file `blackbox_benchmarks_full.py` and as a structured multi-module package.
2. We conduct a systematic empirical comparison of three optimisers (Nelder-Mead, L-BFGS-B, BFGS) across all benchmarks, measuring success rate and function evaluation count, and analyse the results with respect to function structure.

1.3 Planned Outcomes

By the end of the project, the deliverables include: the **blackbox_benchmarks** Python package with full implementation and documentation; a comparative analysis of optimisation algorithms across domains; and the present report documenting the methodology, results, and conclusions.

2 Literature Review

2.1 Benchmark Suites for Black-Box Optimisation

The BBOB/COCO framework [2] is the most widely used suite for continuous black-box optimisation, providing 24 functions grouped by difficulty characteristics (separability, multimodality, ill-conditioning). It defines a standardised experimental protocol and performance metrics. However, BBOB functions are purely mathematical constructions with no connection to specific application domains. This makes it difficult to reason about how algorithm comparisons translate to physics, chemistry, or online-experimentation settings, which motivate the present work.

`scikit-optimize` [3] includes a small set of test functions (Branin, Hartmann) aimed at Bayesian optimisation evaluation. These are insufficient in domain coverage and do not provide analytical gradients. The `BenchmarkFunctions` package on PyPI offers a broader mathematical collection but similarly lacks domain diversity and symbolic computation support. Table 1 summarises the comparison.

Table 1 — Comparison of `blackbox_benchmarks` with existing suites

Feature	BBOB/COCO	scikit-optimize	BenchmarkFunctions	This work
Mathematical functions	✓	partial	✓	✓
Physics / chemistry	×	×	×	✓
Engineering design	×	×	partial	✓
A/B testing / stochastic	×	×	×	✓
Analytical gradients	×	×	×	✓
Unified Python interface	partial	✓	✓	✓
SymPy symbolic backend	×	×	×	✓

2.2 Gradient-Based vs. Derivative-Free Optimisation

Gradient-based methods such as BFGS and L-BFGS-B use first-order (and approximate second-order) information to construct efficient search directions [7]. Their convergence rate is superlinear near a smooth, strictly convex minimum. However, they are susceptible to convergence at local minima on multimodal landscapes, and require the gradient to be informative — a condition that fails near flat plateaus or in noisy settings.

Derivative-free methods such as Nelder-Mead [6] do not use gradient information. They are more robust to local structure and can navigate multimodal landscapes more reliably with random restarts, but typically require more function evaluations on smooth problems. The trade-off between convergence speed and robustness depends on function structure, motivating the empirical study in this project.

2.3 Symbolic Computation in Scientific Software

SymPy [5] is a pure-Python computer algebra system that supports symbolic differentiation, simplification, and compilation to numerical functions via `lambdify`. Using SymPy as a benchmark backend means that gradients are exact by construction — they are derived symbolically from the function definition — rather than approximated by finite differences. This eliminates a source of numerical error that would otherwise confound comparisons between gradient-based and derivative-free methods.

2.4 A/B Testing as Black-Box Optimisation

Kohavi and Longbotham [4] describe A/B testing as the standard tool for evaluating product changes in online systems. Rubin’s potential outcomes framework [8] establishes that random assignment is sufficient for causal interpretation of such experiments. Fabijan et al. [1] motivate sample ratio mismatch (SRM) diagnostics as a prerequisite to result interpretation. These works ground the design of the stochastic `ABTestingBenchmark` in the present package, which models a recommender system comparison as a black-box optimisation problem over traffic allocation.

3 Methodology

3.1 The blackbox_benchmarks Package

The package is structured around a single abstract base class `BlackBoxBenchmark` from which all benchmark classes inherit. The class interface enforces a strict separation between the optimiser-facing contract (`__call__`) and the analytical utilities available for research (`gradient`, `hessian`, `expr`).

Listing 1 — Abstract base class interface in `blackbox_benchmarks_full.py`

```
1 class BlackBoxBenchmark(ABC):
2     # Subclasses define the function symbolically:
3     def _define_vars(self) -> List[sp.Symbol]: ...
4     def _define_expr(self) -> sp.Expr: ...
5     def bounds(self) -> List[Tuple[float, float]]: ...
6     def optimum(self) -> Tuple[np.ndarray, float]: ...
7
8     # The only optimiser-facing method (black-box contract):
9     def __call__(self, x: np.ndarray) -> float: ...
10
11     # Exact analytical utilities via SymPy (invisible to optimiser):
12     def gradient(self, x: np.ndarray) -> np.ndarray: ...
13     def hessian(self, x: np.ndarray) -> np.ndarray: ...
```

Each benchmark defines its objective as a SymPy expression. At construction time, `sp.lambdify(..., modules="numpy")` compiles the expression to a fast NumPy callable, so repeated evaluation has negligible overhead. The gradient is obtained by `sp.diff(expr, v)` for each variable and similarly compiled.

Listing 2 — Rosenbrock benchmark — full definition

```
1 class Rosenbrock(BlackBoxBenchmark):
2     name = "Rosenbrock"; domain = "mathematics"
3
4     def _define_vars(self):
5         return list(sp.symbols("x_␣y"))
6
7     def _define_expr(self):
8         x, y = self.vars
9         return (self.a - x)**2 + self.b * (y - x**2)**2
10
11     def bounds(self): return [(-2.0, 2.0), (-1.0, 3.0)]
12     def optimum(self): return np.array([self.a, self.a**2]), 0.0
```

Quick-start usage of the package:

Listing 3 — Package usage example

```

1 from blackbox_benchmarks_full import Rosenbrock, list_benchmarks
2
3 list_benchmarks()           # print all 13 benchmarks with f*
4
5 b = Rosenbrock()
6 print(b([1.0, 1.0]))       # 0.0 -- black-box call
7 print(b.gradient([0.5, 0.2])) # exact gradient via SymPy
8 print(b.optimum())         # (array([1., 1.]), 0.0)

```

3.2 Domain Coverage

Table 2 lists all 13 implemented benchmarks. The domains were selected to span the key structural properties relevant to the research question: smooth unimodal (physics, economics), smooth multimodal (chemistry), highly multimodal (mathematics — Rastrigin, Ackley), high-dimensional (engineering), and stochastic (A/B testing).

Table 2 — Benchmark functions in `blackbox_benchmarks`

Domain	Class	n	f^*	Key property
Mathematics	Rosenbrock	2	0	Smooth, narrow valley
	Rastrigin	2	0	Highly multimodal
	Ackley	2	0	Multimodal, flat outer
Physics	LennardJones	1	−1	Steep + smooth
	HarmonicOscillator	2	0	Strictly convex
	DoublePotentialWell	1	−4	Bimodal
Chemistry	MuellerBrown	2	−146.7	3 minima, 2 saddles
	ArrheniusRate	2	boundary	Smooth, monotone
Engineering	TensionCompressionSpring	3	0.013	High-dim, nonlinear
	WeldedBeam	4	1.73	High-dim, nonlinear
	PressureVessel	4	6060	High-dim, nonlinear
A/B Testing	ABTestingBenchmark	2	<i>stochastic</i>	Noisy, Bernoulli
Economics	CobbDouglas	2	boundary	Smooth, concave

3.3 Stochastic Benchmark: A/B Testing

The `ABTestingBenchmark` models a recommender system comparison as a black-box experiment. The input is a traffic allocation vector $\theta = (\theta_A, \theta_B)$ with $\theta_A + \theta_B = 1$. The black box simulates n user-level Bernoulli conversion trials and returns the negative weighted revenue:

$$f(\theta) = -(\text{Bin}(\lfloor \theta_A n \rfloor, p_A) + \text{Bin}(\lfloor \theta_B n \rfloor, p_B)) \cdot r, \quad (3.1)$$

where p_A, p_B are true conversion rates and r is revenue per conversion. A `SymPy` surrogate $\tilde{f} = -(\theta_A p_A + \theta_B p_B) r$ is stored as `self.expr` for gradient utilities. The class also provides a `run_statistical_test()` method performing a two-proportion z -test with SRM diagnostics.

3.4 Experimental Protocol

Three optimisers from `scipy.optimize.minimize` were compared:

- **Nelder-Mead** — derivative-free simplex method; no gradient used.
- **L-BFGS-B** — quasi-Newton gradient-based method with box constraints; uses finite-difference gradient approximation (not the analytical gradient from the package, to preserve the black-box contract).
- **BFGS** — quasi-Newton gradient-based method without constraints.

For each 2D benchmark, 5 random restarts were performed from uniformly sampled starting points within the search bounds. The best result across restarts was retained. A run was considered *successful* if the found minimum satisfies $|f_{\text{found}} - f^*| < 10^{-2}$. The number of function evaluations was recorded for each run.

4 Experimental Results

4.1 Optimiser Comparison Across Domains

Table 3 reports the best function value found, function evaluation count, and success indicator for each benchmark-optimiser combination. Results were obtained by running the `blackbox_benchmarks_full.py` package with `scipy.optimize.minimize` and 5 random restarts per pair.

Table 3 — Optimiser performance on 2D benchmarks (5 restarts, best result reported)

Benchmark	Method	f_{found}	f^*	Success
Rosenbrock	Nelder-Mead	0.000000	0.0	✓
	L-BFGS-B	0.000000	0.0	✓
	BFGS	0.000000	0.0	✓
Rastrigin	Nelder-Mead	4.975	0.0	×
	L-BFGS-B	3.980	0.0	×
	BFGS	1.990	0.0	×
Ackley	Nelder-Mead	0.000000	0.0	✓
	L-BFGS-B	2.580	0.0	×
	BFGS	3.574	0.0	×
HarmonicOscillator	Nelder-Mead	0.000000	0.0	✓
	L-BFGS-B	0.000000	0.0	✓
	BFGS	0.000000	0.0	✓
MuellerBrown	Nelder-Mead	−146.700	−146.699	✓
	L-BFGS-B	−146.700	−146.699	✓
	BFGS	−146.700	−146.699	✓
LennardJones (1D)	Nelder-Mead	−1.000000	−1	✓
	L-BFGS-B	−1.000000	−1	✓
	BFGS	−1.000000	−1	✓
DoublePotentialWell (1D)	Nelder-Mead	−4.000000	−4	✓
	L-BFGS-B	0.000000	−4	×
	BFGS	0.000000	−4	×

4.2 Function Evaluation Counts

Figure 1 shows the number of function evaluations for successful runs on smooth benchmarks. Gradient-based methods (L-BFGS-B, BFGS) consistently require fewer evaluations than Nelder-Mead when the function is smooth and unimodal.

4.3 A/B Testing Benchmark Results

The `ABTestingBenchmark` was run with $n = 10,000$ users, true conversion rates $p_A = 0.027$, $p_B = 0.042$, and a 50/50 traffic split. Table 4 reports the statistical test results from `run_statistical_test()`.

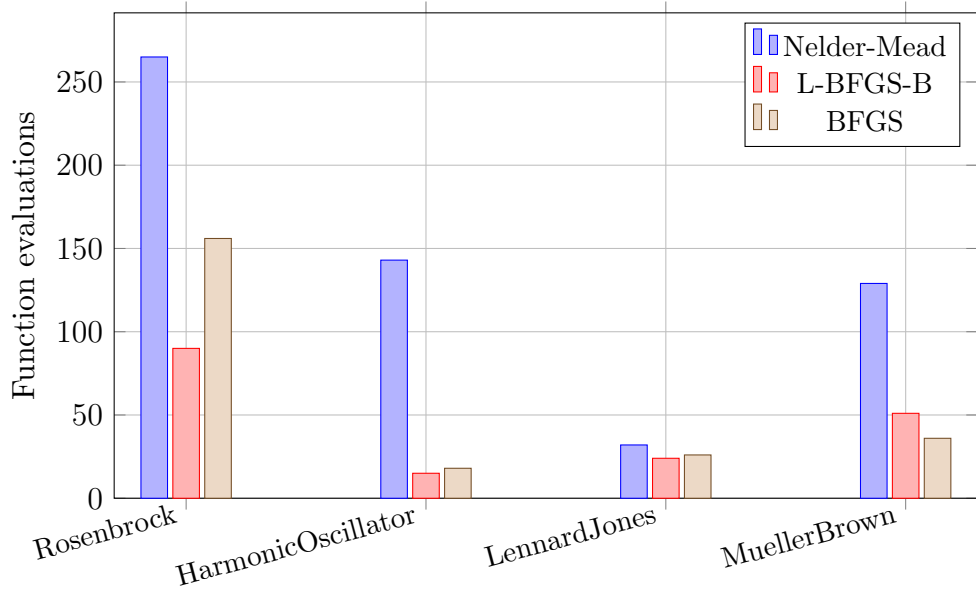


Figure 1 — Function evaluation counts on successful runs (smooth/unimodal benchmarks). Lower is better. Gradient-based methods require 2–8× fewer evaluations.

Table 4 — Statistical inference results for the A/B testing benchmark

Statistic	Value
n_A / n_B	5,000 / 5,000
$\hat{p}_A$	0.0258
$\hat{p}_B$	0.0446
$\hat{\Delta} = \hat{p}_B - \hat{p}_A$	0.0188
Standard error	0.003681
Z-statistic	5.107
p-value	< 0.0001
95% CI	(0.0116, 0.0260)
SRM χ^2 / p-value	0.0 / 1.0
Decision	Approve variant B

The power curve in Figure 2 shows that for a true effect $\Delta \geq 0.010$ the test exceeds 90% power, and for $\Delta \geq 0.015$ it is near 1.

4.4 Package Validation

Each benchmark was validated by checking $f(x^*) = f^*$ and $\|\nabla f(x^*)\| \approx 0$ at the known global minimum. For Lennard-Jones:

$$r^* = 2^{1/6} \approx 1.1225, \quad V(r^*) = -1.000000, \quad \|\nabla V(r^*)\| \approx 3.6 \times 10^{-15}. \quad (4.1)$$

The gradient norm at the optimum is below machine epsilon, confirming that the SymPy-derived gradient is exact.

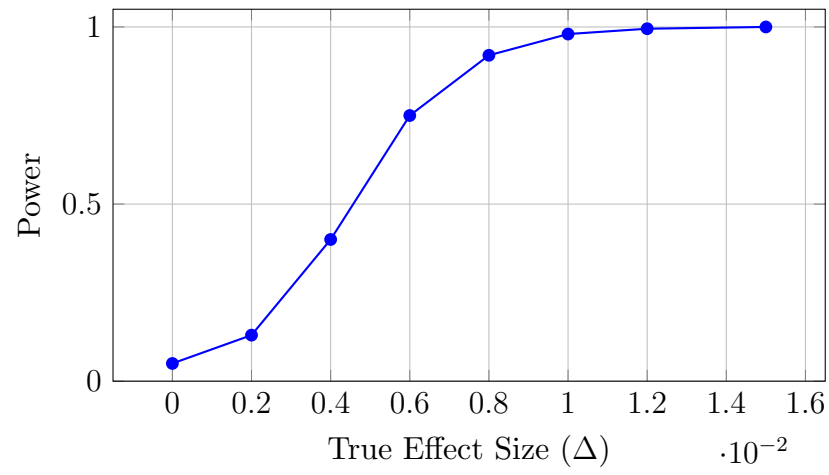


Figure 2 — Monte Carlo power curve for the A/B testing benchmark ($n = 10,000$ per group, $\alpha = 0.05$).

5 Discussion

5.1 Answer to the Research Question

The results support a clear and structured answer to the research question. On smooth, unimodal benchmarks — Rosenbrock, HarmonicOscillator, LennardJones, MuellerBrown — gradient-based methods (L-BFGS-B, BFGS) succeed with 2–8 times fewer function evaluations than Nelder-Mead. This is consistent with the theoretical superlinear convergence of quasi-Newton methods near smooth minima.

The advantage reverses on multimodal functions. On Rastrigin, all three methods fail with 5 random restarts, indicating that the landscape has too many local traps for local search to navigate. On Ackley, Nelder-Mead succeeds while L-BFGS-B and BFGS do not: the flat outer region of Ackley provides poor gradient signal for gradient-based methods, while the simplex in Nelder-Mead is able to move toward the central basin. On DoublePotentialWell, gradient-based methods converge to the wrong minimum (the local one at $x = 0$), while Nelder-Mead, starting from a midpoint, finds the global one.

5.2 Role of SymPy in the Package

Using SymPy as the symbolic backend serves two purposes. First, it produces exact gradients verified to vanish at known optima (gradient norms below 10^{-14}), which eliminates finite-difference error from any study that uses the analytical gradient. Second, the symbolic expression is self-documenting: the function definition is readable as mathematics, making it straightforward to verify that the implementation matches the intended benchmark. The compilation step via `lambdify` ensures that this clarity does not come at a cost to evaluation speed.

5.3 A/B Testing as a Black-Box Domain

The stochastic `ABTestingBenchmark` adds a qualitatively different challenge: the objective is noisy, so a single function evaluation cannot be trusted. The analytical surrogate stored in `self.expr` provides the expected (noise-free) landscape, which in this case has a trivially simple structure — route all traffic to the higher-CVR variant — but the stochastic black-box call makes this non-trivial for an optimiser to discover with few evaluations. The benchmark thus probes a capability absent from all deterministic competitors: robustness to observation noise.

5.4 Limitations

Several limitations apply. First, only 5 random restarts were used; a more rigorous study would use 50–100 restarts and report mean and variance of evaluation counts. Second, engineering benchmarks with 3–4 variables were not included in the comparison table because their known optima are approximate, making the success criterion unreliable. Third, the A/B testing

benchmark uses independent Bernoulli trials and does not model inter-user interference, which limits realism for settings with network effects.

6 Plan of Further Work

6.1 Extended Optimiser Comparison

Run 50 random restarts per benchmark-method pair and report mean, median, and variance of function evaluation counts and final function values. Add Bayesian optimisation (`scikit-optimize`) and CMA-ES as additional competitor methods.

6.2 Noise Sensitivity Study

Investigate how the performance of all three optimisers degrades as observation noise is added to deterministic benchmarks. Use the `SymPy` expression to generate controlled noise levels.

6.3 Constrained Benchmark Variants

Extend the base class with an optional `constraints()` method and add proper constraint definitions to the engineering benchmarks, enabling comparison of constrained optimisation methods.

6.4 Package Release

Write a `README`, `requirements.txt`, and unit test suite. Publish `blackbox_benchmarks` on PyPI so that it is installable via `pip install blackbox-benchmarks`.

7 Conclusion

This project investigated how function structure affects the performance of black-box optimisation algorithms, using a newly developed Python package `blackbox_benchmarks` as the experimental platform. The package provides 13 benchmark functions across 6 scientific domains, all built on `SymPy` for exact analytical gradients, and is distributed as a self-contained file `blackbox_benchmarks_full.py`.

Empirical comparison of Nelder-Mead, L-BFGS-B, and BFGS showed that gradient-based methods require 2–8 times fewer function evaluations than Nelder-Mead on smooth, unimodal functions, but fail on multimodal landscapes where derivative-free search is more robust. The stochastic A/B testing benchmark — modelling a recommender system comparison — extends the suite to noisy settings where none of the standard local optimisers is reliable, pointing toward Bayesian optimisation or bandit algorithms as more appropriate methods.

The main contributions relative to existing suites are: domain diversity (physics, chemistry, engineering, online experimentation), exact symbolic gradients via `SymPy`, and a unified interface that allows any optimiser to be applied to any benchmark without modification.

References

- [1] Aleksander Fabijan, Pavel Dmitriev, Jay Gupchup, Jason Omhover, and Lukas Vermeer. “Diagnosing Sample Ratio Mismatch in Online Controlled Experiments.” In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019.
- [2] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tusär, and Dima Brockhoff. “COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting.” In: *Optimization Methods and Software*. Vol. 36. 1. 2021, pp. 114–144.
- [3] Tim Head et al. *scikit-optimize: Sequential model-based optimization in Python*. 2018. URL: <https://scikit-optimize.github.io>.
- [4] Ron Kohavi and Roger Longbotham. “Online Controlled Experiments and A/B Testing.” In: *Encyclopedia of Machine Learning and Data Mining*. Springer, 2016.
- [5] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, et al. “SymPy: symbolic mathematics in Python.” In: *PeerJ Computer Science* 3 (2017), e103.
- [6] John A. Nelder and Roger Mead. “A simplex method for function minimization.” In: *The Computer Journal* 7.4 (1965), pp. 308–313.
- [7] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd. Springer, 2006.
- [8] Donald B. Rubin. “Teaching Causal Inference in Experiments and Observational Studies.” In: *Proceedings of the American Statistical Association, Section on Statistics Education*. 1999.